

ASSIGNMENT – 6

Name: T.Sathvik

Roll_no: 2503a52415

Batch_no: 14

Task-1: (Classes-Data Validation)

Prompt:

Create a Python Student class with a constructor that has three attributes: name, roll _ no, and marks. Add a method called is _pass () that checks whether the student has passed. The student should pass if marks are greater than or equal to 40. Include sample objects and display their details and pass/fail status. Provide clear and simple Python code with an explanation.

Input:

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks

    def is_pass(self):
        return self.marks >= 40

s1 = Student("Sathvik", "2503A52415", 68)
s2 = Student("Rahul", "2503A52416", 32)

print("Name:", s1.name)
print("Roll No:", s1.roll_no)
print("Marks:", s1.marks)
print("Result:", "Pass" if s1.is_pass() else "Fail")

print()
print("Name:", s2.name)
print("Roll No:", s2.roll_no)
print("Marks:", s2.marks)
print("Result:", "Pass" if s2.is_pass() else "Fail")
```

Output:

```
Name: Sathvik
Roll No: 2503A52415
Marks: 68
Result: Pass

Name: Rahul
Roll No: 2503A52416
Marks: 32
Result: Fail
```

Example:

The class stores the student's name, roll number and marks. The is_pass() method checks whether marks are 40 or more.

Task-2: (Loops: Pattern Generation)

Prompt:

Write a Python function that generates and prints a right-angled triangle star pattern using a for and while loop. The function should accept the number of rows as input. For example, if the number of rows is 5, the output should contain 1 star in the first row, 2 stars in the second row, and continue until 5 stars in the fifth row. Provide the code and explain how the loop works.

Input:

```
def triangle_for(n):
    for i in range(1, n + 1):
        print("'" * i)

print("Using for loop:")
triangle_for(5)

def triangle_while(n):
    i = 1
    while i <= n:
        print("'" * i)
        i = i + 1

print("Using while loop:")
triangle_while(5)
```

Output:

```
Using for loop:
*
**
***
****
*****

Using while loop:
*
**
***
****
*****
```

Example:

The for loop prints one more star in every row. The while loop also prints the stars by increasing i after every row.

Task-3: (Conditional Statements: Number Analysis)**Prompt:**

Write a Python function called `check_number(num)` that determines whether a given number is positive, negative, or zero using if-elif-else statements. Test the function with multiple values, including a positive number, a negative number, and zero. Display the results clearly and explain the decision-making logic.

Input:

```
def check_number(num):
    if num > 0:
        return "Positive"
    elif num < 0:
        return "Negative"
    else:
        return "Zero"

print(25, check_number(25))
print(-10, check_number(-10))
print(0, check_number(0))
print(7, check_number(7))
print(-3, check_number(-3))
```

Output:

```
25 Positive
-10 Negative
0 Zero
7 Positive
-3 Negative
```

Example:

The if statement checks for a positive number. The elif checks for a negative number. If neither is true, the number is zero.

Task-4: (Nested Conditionals: Discount Eligibility)

Prompt:

Write a Python function called `check_discount(age, is_member)` using nested if statements. If the person's age is greater than or equal to 60, they should be eligible for a senior discount. If the person is also a member, they should receive an additional member discount. If the person is below 60, check separately whether they are a member and display the appropriate discount eligibility. Test the function with multiple combinations of age and membership status. Explain the decision flow.

Input:

```
def check_discount(age, is_member):
    if age >= 60:
        print("Senior discount eligible")
        if is_member:
            print("Additional member discount")
        else:
            print("No member discount")
    else:
        print("No senior discount")
        if is_member:
            print("Member discount eligible")
        else:
            print("No discount")

print("Customer 1:")
check_discount(65, True)

print("Customer 2:")
check_discount(65, False)

print("Customer 3:")
check_discount(30, True)

print("Customer 4:")
check_discount(30, False)
```

Output:

```
Customer 1:
Senior discount eligible
Additional member discount
Customer 2:
Senior discount eligible
No member discount
Customer 3:
No senior discount
Member discount eligible
Customer 4:
No senior discount
No discount
```

Example:

The first if checks the age. Inside it, another if checks membership. For people below 60, the second part checks whether they are members.

Task-5: (Class: Circle Mathematical Operations)

Prompt:

Create a Python Circle class with a constructor that accepts radius as an attribute. Add one method called `area()` to calculate the area of the circle and another method called `circumference()` to calculate its circumference. Use the formulas $\text{Area} = \pi \times r^2$ and $\text{Circumference} = 2 \times \pi \times r$. Create a Circle object with a sample radius, calculate both values, display the results, and provide a clear explanation of the code.

Input:

```
import math

class Circle:
    def __init__(self, radius):
```

```
        self.radius = radius

    def area(self):
        return math.pi * self.radius * self.radius

    def circumference(self):
        return 2 * math.pi * self.radius

c = Circle(5)

print("Radius:", c.radius)
print("Area:", round(c.area(), 2))
print("Circumference:", round(c.circumference(), 2))
```

Output:

```
Radius: 5
Area: 78.54
Circumference: 31.42
```

Example:

The Circle class stores the radius. The `area()` method calculates $\pi \times r \times r$, and `circumference()` calculates $2 \times \pi \times r$.